MARKET SEGMENTATION. PCA+Clustering

```
set.seed(4801)
all_listings<- readRDS("all_listings.rds")
amen <- readRDS("amenities_sparse.rds")
```

Are there performance-based segments of listings that share similar pricing levels, quality signals, and booking patterns?

We will focus on analyzing this answering the following two questions:

1. Can we infer meaningful similarities between listings based solely on their amenity profiles?
2. Can we identify clusters of listings with similar price-quality-occupancy profiles?

Can we infer meaningful similarities between listings based solely on their amenity profiles?

PCA

Given that the *amenities_sparse.rds* is a sparse binary matrix, we will make use of the **irlba** package for performing PCA.

```
library(irlba)
```

```
## Warning: package 'irlba' was built under R version 4.3.3
```

```
## Loading required package: Matrix
```

```
## Warning: package 'Matrix' was built under R version 4.3.2
```

```
# Number of components to extract
k <- 20

  # Sparse PCA via truncated SVD
set.seed(4801)
amen_pca <- irlba(amen, nv = k, center = TRUE, scale = FALSE)

amen_scores <- amen %*% amen_pca$v
```

Approximate explained variance

```r
var_explained <- amen_pca$d^2 / sum(amen_pca$d^2)
var_explained
```

```
##  [1] 0.33567259 0.11325991 0.06801670 0.05221236 0.05008997 0.04502409
##  [7] 0.03658619 0.03179953 0.02943731 0.02820503 0.02436876 0.02380808
## [13] 0.02338294 0.02208475 0.02140081 0.02014905 0.01948831 0.01864203
## [19] 0.01833211 0.01803949
```

We just came to realize that with 6PC's we explain 2/3 of the variation. Let's try with it.

```r
sum(var_explained[1:6])
```

```
## [1] 0.6642756
```

```r
amen_pca <- irlba(amen, nv = 6, center = TRUE, scale = FALSE)
```

SIGN FIXING For reproducibility

```r
rownames(amen_pca$v) <- colnames(amen)
loadings <- amen_pca$v

ref_vars_amen <- c("wifi","heating","self check-in","bbq grill",
                   "kitchen","fire extinguisher")

for(j in seq_along(ref_vars_amen)){
  v <- ref_vars_amen[j]
  if(v %in% rownames(loadings)){
    if(loadings[v, j] < 0) loadings[, j] <- -loadings[, j]
  }
}

amen_pca$v <- loadings
amen_scores <- amen %*% amen_pca$v
```

```r
rownames(amen_pca$v) <- colnames(amen)
loadings <- amen_pca$v
top_loadings <- function(component, n = 5) {
  x <- loadings[, component]
  idx <- order(x, decreasing = TRUE)
  list(
    top_positive = names(x[idx[1:n]]),
    top_negative = names(x[idx[(length(x)-n+1):length(x)]])
  )
}
```

```r
top_loadings(1)
```

LOADINGS INTERPRETATIONS

```
## $top_positive
## [1] "air conditioning"     "tv"                    "washer"
## [4] "wifi"                 "lock on bedroom door"
##
## $top_negative
## [1] "toaster"           "dining table"      "wine glasses"
## [4] "cleaning products" "freezer"
```

PC1 summarizes Modern Comfort vs. Basic Interior Setup.

```
top_loadings(2)
```

```
## $top_positive
## [1] "heating"      "coffee maker" "oven"         "stove"        "dryer"
##
## $top_negative
## [1] "coffee"                 "cleaning products"      "free dryer \\u in unit"
## [4] "outdoor dining area"    "central heating"
```

PC2 encapsulates Indoor Heating & Essentials vs. Outdoor/Shared Features.

```
top_loadings(3)   #
```

```
## $top_positive
## [1] "essentials"          "hangers"             "free street parking"
## [4] "self check-in"       "iron"
##
## $top_negative
## [1] "outdoor dining area"      "air conditioning"
## [3] "tv"                       "free parking on premises"
## [5] "washer"
```

PC3 represents Urban Self-Service Units vs. Home-Style Amenities.

```
top_loadings(4)
```

```
## $top_positive
## [1] "bbq grill"           "outdoor dining area" "outdoor furniture"
## [4] "dishwasher"          "pets allowed"
##
## $top_negative
## [1] "cleaning products" "clothing storage"  "shower gel"
## [4] "body soap"         "conditioner"
```

PC4 interprets Outdoor & Pet-Friendly vs. Indoor Personal-Care Focus.

```
top_loadings(5)
```

```
## $top_positive
## [1] "dishwasher"     "oven"           "cooking basics" "kitchen"
## [5] "dining table"
##
## $top_negative
## [1] "luggage dropoff allowed" "heating"
## [3] "shampoo"                 "fire extinguisher"
## [5] "first aid kit"
```

PC5 summarizes Full Kitchen Functionality vs. Host Safety/Essentials.

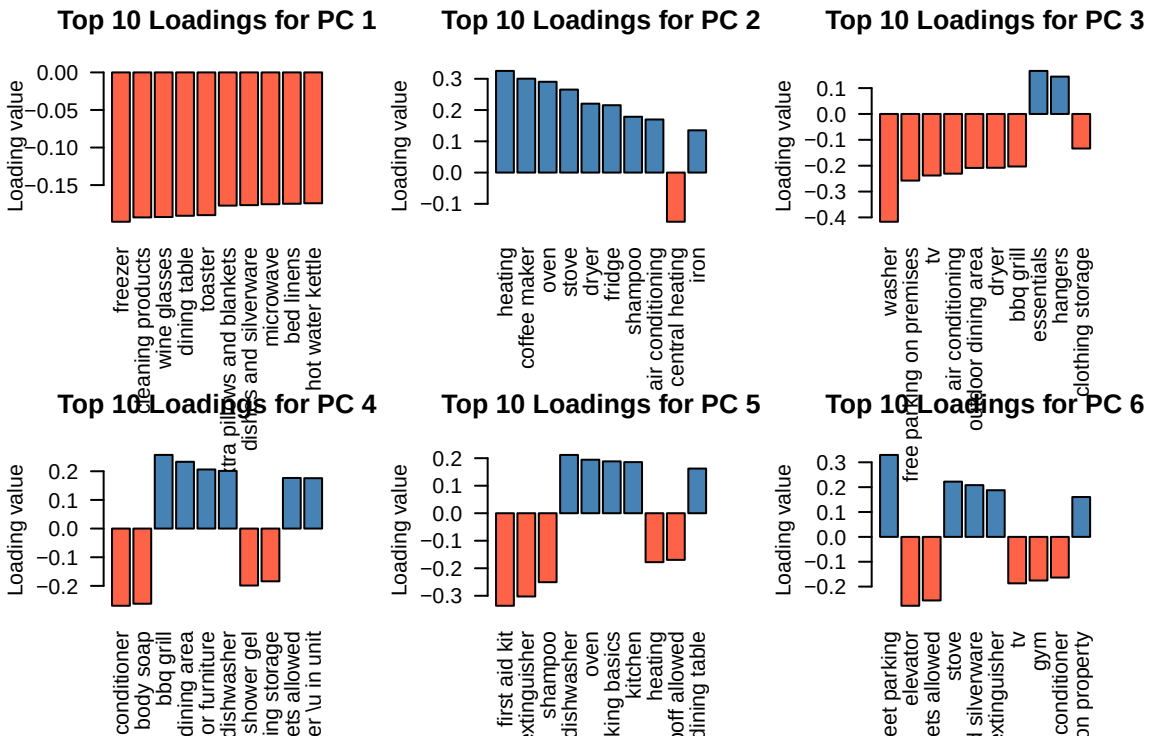
```
top_loadings(6)
```

```
## $top_positive
## [1] "free street parking"
## [2] "stove"
## [3] "dishes and silverware"
## [4] "fire extinguisher"
## [5] "exterior security cameras on property"
##
## $top_negative
## [1] "conditioner"  "gym"          "tv"           "pets allowed" "elevator"
```

PC6 represents Practical/Security Amenities vs. Luxury Building Features.

```
loadings <- amen_pca$v      # PCA loadings
rownames(loadings) <- colnames(amen)
```

```
plot_top_loadings <- function(pc, n = 10) {
  x <- loadings[, pc]
  idx <- order(abs(x), decreasing = TRUE)[1:n]
  barplot(
    x[idx],
    las = 2,
    main = paste("Top", n, "Loadings for PC", pc),
    col = ifelse(x[idx] > 0, "steelblue", "tomato"),
    ylab = "Loading value"
  )
}
par(mfrow=c(2,3))
for (i in 1:6) plot_top_loadings(i)
```



VISUALIZATION

```
par(mfrow=c(1,1))
```

```
set.seed(123)
idx <- sample(1:nrow(amen), 3)
idx
```

CHECK INTERPRETABILITY

```
## [1] 51663 57870  2986
```

```
get_amenities <- function(i){
  idx <- which(amen[i, ] == 1)
  colnames(amen)[idx]
}
amenities_1 <- get_amenities(idx[1])
amenities_2 <- get_amenities(idx[2])
amenities_3 <- get_amenities(idx[3])
amen_scores <- amen %*% amen_pca$v
pca1 <- amen_scores[idx[1], ]
pca2 <- amen_scores[idx[2], ]
pca3 <- amen_scores[idx[3], ]
```

```
amenities_1
```

```
##  [1] "smoke alarm"
```

```
##  [2] "kitchen"
##  [3] "carbon monoxide alarm"
##  [4] "hot water"
##  [5] "hangers"
##  [6] "essentials"
##  [7] "hair dryer"
##  [8] "iron"
##  [9] "dishes and silverware"
## [10] "cooking basics"
## [11] "bed linens"
## [12] "microwave"
## [13] "fridge"
## [14] "dedicated worksp air conditioninge"
## [15] "shampoo"
## [16] "fire extinguisher"
## [17] "self check-in"
## [18] "heating"
## [19] "first aid kit"
## [20] "free street parking"
## [21] "extra pillows and blankets"
## [22] "free parking on premises"
## [23] "private entrance"
## [24] "toaster"
## [25] "wine glasses"
## [26] "body soap"
## [27] "conditioner"
## [28] "room-darkening shades"
## [29] "outdoor furniture"
## [30] "luggage dropoff allowed"
## [31] "clothing storage"
## [32] "keypad"
## [33] "central air conditioning"
## [34] "portable fans"
## [35] "fire pit"
## [36] "mini fridge"
## [37] "coffee maker: keurig coffee m air conditioninghine"
## [38] "wifi \\u"
## [39] "safe"
## [40] "shared patio or balcony"
## [41] "garden view"
## [42] "free dryer \\u in building"
## [43] "free washer \\u in building"
## [44] "ev charger"
## [45] "tv with streaming, streaming"
```

```
pca1
```

```
## [1] -4.1145401  1.1211444  0.4969188  0.3240133 -2.0283082  0.6949847
```

PC1 high; comfort amenities. PC4 slightly high; outdoor features. PC5 low; safety items included Outdoor-friendly, suburban, family-ready home

```
amenities_2
```

```
##  [1] "smoke alarm"
##  [2] "wifi"
##  [3] "kitchen"
##  [4] "carbon monoxide alarm"
##  [5] "hot water"
##  [6] "hangers"
##  [7] "essentials"
##  [8] "hair dryer"
##  [9] "iron"
## [10] "dishes and silverware"
## [11] "cooking basics"
## [12] "bed linens"
## [13] "microwave"
## [14] "fridge"
## [15] "dedicated worksp air conditioninge"
## [16] "shampoo"
## [17] "self check-in"
## [18] "first aid kit"
## [19] "long term stays allowed"
## [20] "free street parking"
## [21] "extra pillows and blankets"
## [22] "cleaning products"
## [23] "freezer"
## [24] "dining table"
## [25] "dishwasher"
## [26] "exterior security cameras on property"
## [27] "private entrance"
## [28] "toaster"
## [29] "shower gel"
## [30] "hot water kettle"
## [31] "wine glasses"
## [32] "body soap"
## [33] "conditioner"
## [34] "pets allowed"
## [35] "room-darkening shades"
## [36] "outdoor furniture"
## [37] "luggage dropoff allowed"
## [38] "baking sheet"
## [39] "laundromat nearby"
## [40] "blender"
## [41] "free dryer \\u in unit"
## [42] "single level home"
## [43] "books and reading material"
## [44] "free washer \\u in unit"
## [45] "ethernet connection"
## [46] "smart lock"
## [47] "clothing storage: closet"
## [48] "coffee maker: keurig coffee m air conditioninghine"
## [49] "stainless steel oven"
## [50] "air conditioning - split type ductless system"
## [51] "heating - split type ductless system"
```

```
## [52] "stainless steel gas stove"
## [53] "tv with streaming, streaming, streaming"
## [54] "free driveway parking on premises \\u sp air conditioninge"
## [55] "ev charger - level"
## [56] "shared b air conditioningkyard \\u not fully fenced"
```

```
pca2
```

```
## [1] -5.81757682  0.20801886  0.37861416  0.07175762 -0.45239769 -0.25099494
```

PC1 very high; luxury & complete. PC3 slightly high; self-check-in/urban. PC4 neutral; not strongly outdoor High-comfort, smart-tech, fully equipped suburban home

```
amenities_3
```

```
##  [1] "smoke alarm"                          "wifi"
##  [3] "carbon monoxide alarm"                "hangers"
##  [5] "essentials"                           "hair dryer"
##  [7] "iron"                                 "bed linens"
##  [9] "dedicated worksp air conditioninge"   "shampoo"
## [11] "air conditioning"                     "tv"
## [13] "fire extinguisher"                    "self check-in"
## [15] "heating"                              "first aid kit"
## [17] "extra pillows and blankets"           "coffee maker"
## [19] "pets allowed"                         "building staff"
## [21] "paid parking off premises"
```

```
pca3
```

```
## [1] -1.357736  1.733639  0.218553  0.314137 -2.062923 -0.738737
```

PC1 medium; modern comfort. PC2 high; strong indoor essentials. PC5 very low; no kitchen. PC6 low; building-level amenities Urban minimalist / hotel-style unit

K-MEANS

```
set.seed(123)

results <- lapply(3:20, function(k){
  km <- kmeans(amen_scores, centers = k, nstart = 50, iter.max = 1000)
  km$tot.withinss
})
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

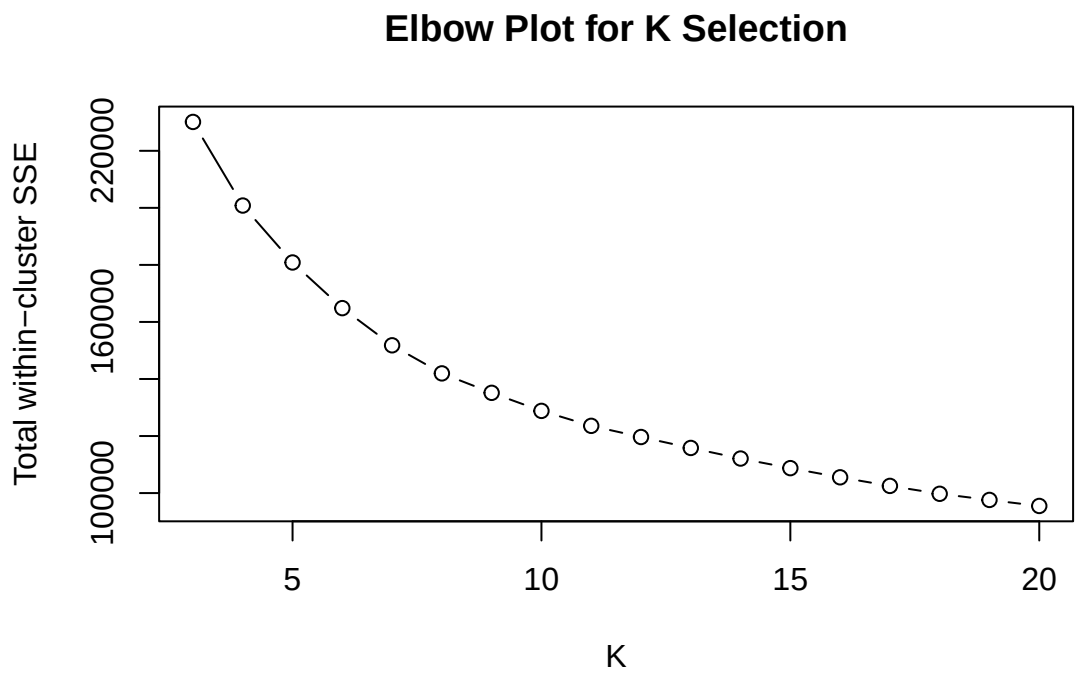
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

For deciding K (number of clusters) we perform Elbow plot and Silhouette score analysis.

```
plot(3:20, unlist(results), type="b", xlab="K", ylab="Total within-cluster SSE", main = "Elbow Plot for
```



ELBOW PLOT

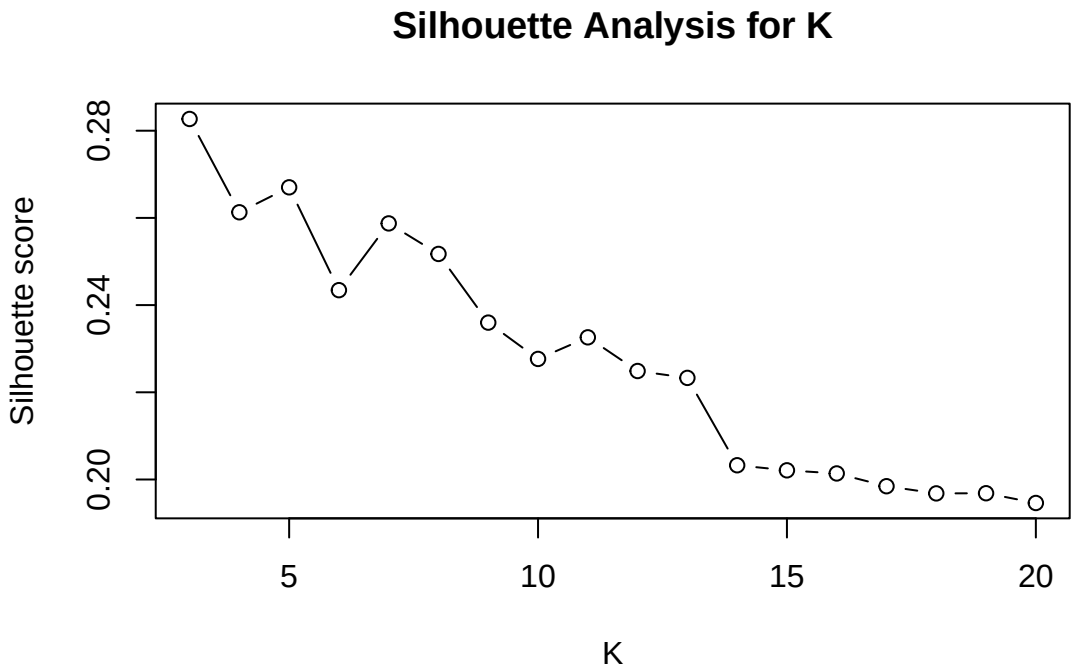
```
library(cluster)
set.seed(123)

idx <- sample(1:nrow(amen_scores), 3000)
sub_scores <- amen_scores[idx, ]

sil <- sapply(3:20, function(k){
  km <- kmeans(sub_scores, centers=k, nstart=50, iter.max = 1000)
```

```
  d <- dist(sub_scores)
  ss <- silhouette(km$cluster, d)
  mean(ss[,3])
})

plot(3:20, sil, type="b", xlab="K", ylab="Silhouette score", main = "Silhouette Analysis for K")
```



SILHOUETTE SCORE

We choose K=8. None of the plots gave any satsifactory criteria for determining a K, and this is the best approximation.

```
K <- 8
```

```
set.seed(123)
kfit <- kmeans(amen_scores, centers = K, nstart = 50, iter.max = 1000)

cluster_labels <- kfit$cluster

amen_scores_mat <- as.matrix(amen_scores)
cluster_pca_means <- aggregate(amen_scores_mat,
                               by = list(cluster = cluster_labels),
                               FUN = mean)
cluster_pca_means
```

Proceeding with this K

```
##   cluster         V1        V2          V3          V4           V5          V6
## 1       1 -2.7150117 2.8434438 -0.15086392  0.72583033 -0.604490510  0.70407336
## 2       2 -1.7088895 1.5370396 -0.08974617  0.08507711 -1.328534191 -0.27653081
## 3       3 -2.3267214 3.4769686  0.56060124  1.44035764  0.003719351 -1.73723296
## 4       4 -3.0938493 1.7380944 -0.46592845 -0.06762378  0.327718613  0.09973618
## 5       5 -4.6986256 2.3563691 -1.54741334 -0.12512283 -0.570414843  0.00492698
## 6       6 -0.4603436 0.6548983 -0.94723976  0.29407126 -0.352726855  0.11751836
## 7       7 -5.5619834 0.3997399 -0.53124924  0.89754059 -0.470726158 -0.03425223
## 8       8 -4.4434516 1.0127611  0.12899627 -0.24307386 -0.511200053  0.17355215
```

```
all_listings$amen_cluster <- cluster_labels
```

We add this new variable.

```
library(reshape2)
```

VISUALIZATION

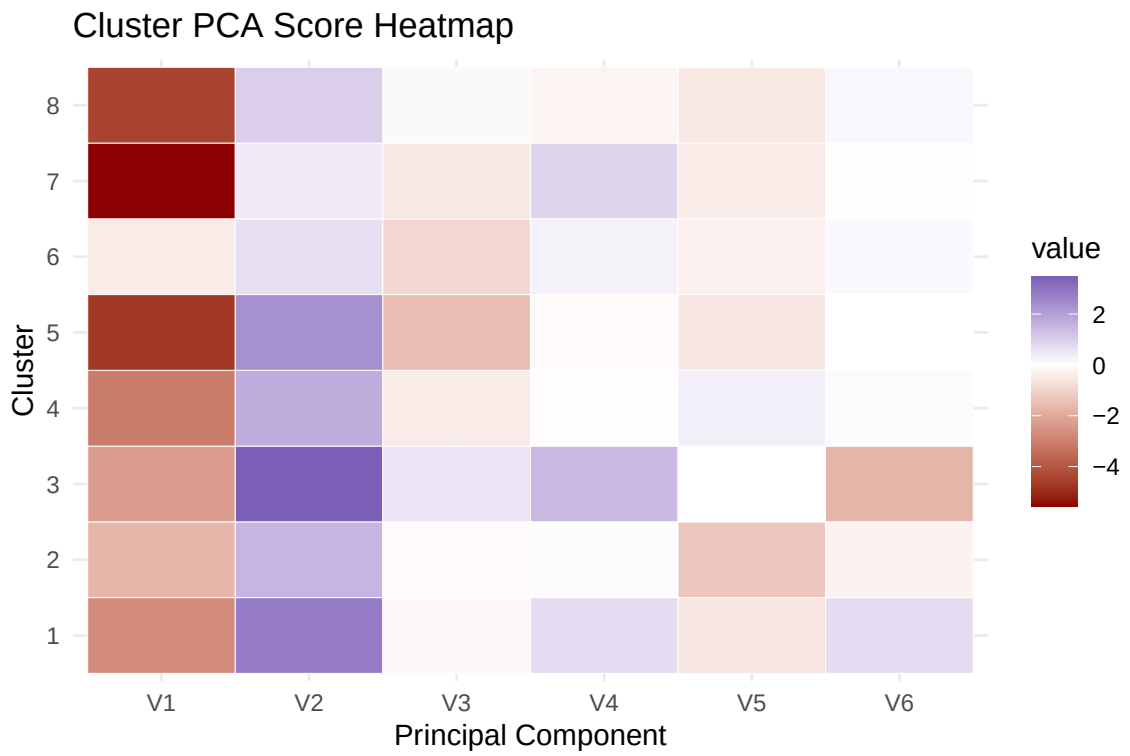
```
## Warning: package 'reshape2' was built under R version 4.3.2
```

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.3.3
```

```
df_long <- melt(cluster_pca_means, id.vars="cluster")

ggplot(df_long, aes(x = variable, y = factor(cluster), fill = value)) +
  geom_tile(color = "white") +
  scale_fill_gradient2(low="darkred", mid="white", high="darkblue",
                       midpoint=0) +
  theme_minimal() +
  labs(title="Cluster PCA Score Heatmap",
       x="Principal Component",
       y="Cluster")
```



Can we identify clusters of listings with similar price-quality-occupancy profiles?

CHOOSING AND TRANSFORMING DATA

```
set.seed(4801)
library(fastDummies)
```

```
## Warning: package 'fastDummies' was built under R version 4.3.3
```

```
library(dplyr)
```

```
## Warning: package 'dplyr' was built under R version 4.3.2
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
df <- all_listings %>%
  select(
    log_price,
    occupancy_rate,
    number_of_reviews,
    reviews_per_month,
    review_scores_rating,
    amenity_count,
    accommodates,
    bedrooms,
    beds,
    bathrooms,
    host_is_superhost,
    host_listings_count,
    host_identity_verified,
    availability_365
  )

df <- scale(df)
```

PCA Part

```r
pca_list <- prcomp(df, center = TRUE, scale. = FALSE)
```

```r
fix_signs <- function(pca_obj, ref_vars){
  L <- pca_obj$rotation
  S <- pca_obj$x

  for (j in seq_along(ref_vars)) {
    v <- ref_vars[j]
    if (v %in% rownames(L)) {
      if (L[v, j] < 0) {
        L[, j] <- -L[, j]
        S[, j] <- -S[, j]
      }
    }
  }

  pca_obj$rotation <- L
  pca_obj$x <- S
  return(pca_obj)
}

ref_vars <- c("log_price",
              "number_of_reviews",
              "host_is_superhost",
              "occupancy_rate",
              "review_scores_rating",
              "availability_365")
```

```r
pca_list <- fix_signs(pca_list, ref_vars)
```

SIGN FIXING

```r
summary(pca_list)
```

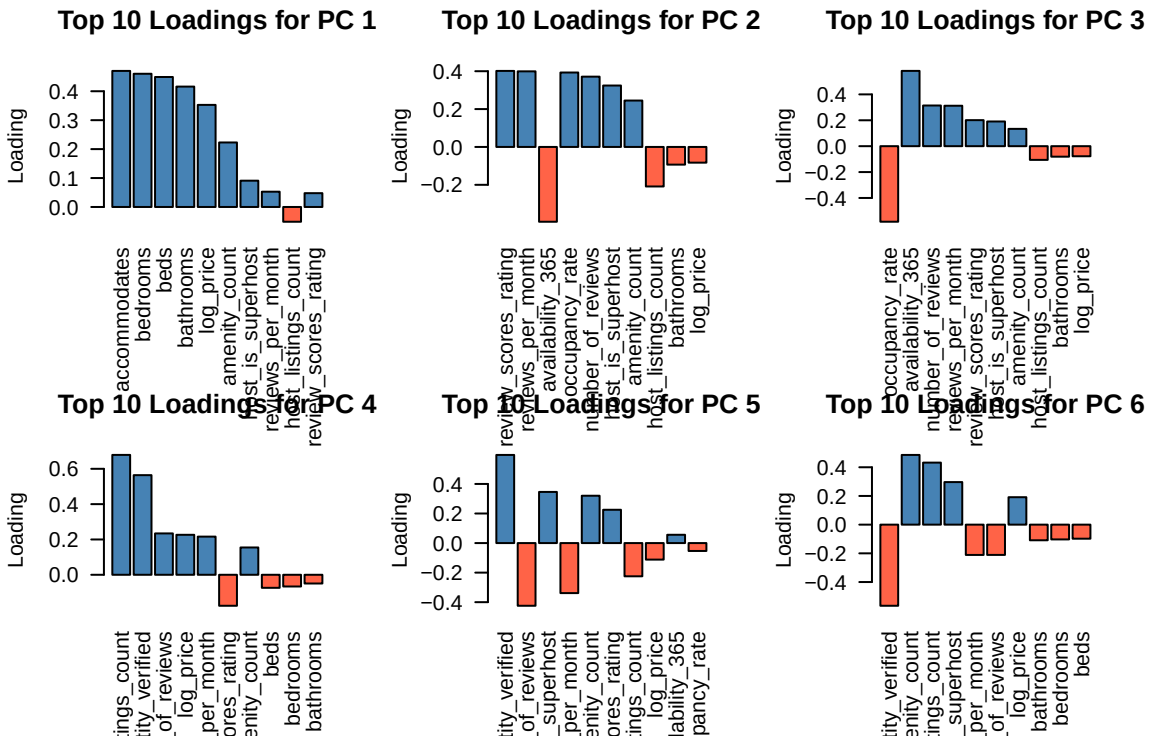
LOADINGS INTERPRETATIONS

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     1.9143 1.6203 1.3113 1.04457 0.99646 0.95852 0.82601
## Proportion of Variance 0.2618 0.1875 0.1228 0.07794 0.07092 0.06563 0.04873
## Cumulative Proportion  0.2618 0.4493 0.5721 0.65006 0.72098 0.78661 0.83534
##                           PC8    PC9    PC10    PC11    PC12    PC13    PC14
## Standard deviation     0.74892 0.7256 0.65487 0.60932 0.47105 0.43756 0.06654
## Proportion of Variance 0.04006 0.0376 0.03063 0.02652 0.01585 0.01368 0.00032
## Cumulative Proportion  0.87540 0.9130 0.94364 0.97016 0.98601 0.99968 1.00000
```

Let's choose again 6 principal components (almost 80% of variance is explained)

```r
plot_top_loadings <- function(pca_obj, pc = 1, n = 10) {
  loadings <- pca_obj$rotation[, pc]
  idx <- order(abs(loadings), decreasing = TRUE)[1:n]

  barplot(loadings[idx],
          las = 2,
          col = ifelse(loadings[idx] > 0, "steelblue", "tomato"),
          main = paste("Top", n, "Loadings for PC", pc),
          ylab = "Loading")
}
par(mfrow=c(2,3))
for (i in 1:6) plot_top_loadings(pca_list, pc = i, n = 10)
```




```
par(mfrow=c(1,1))
```

With these 6 PCs

```
scores_list <- pca_list$x[, 1:6]
```

K-MEANS

```
set.seed(123)
k_range <- 3:12

wcss <- sapply(k_range, function(k) {
  kmeans(scores_list, centers = k, nstart = 50, iter.max = 1000)$tot.withinss
})
```

ELBOW PLOT

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

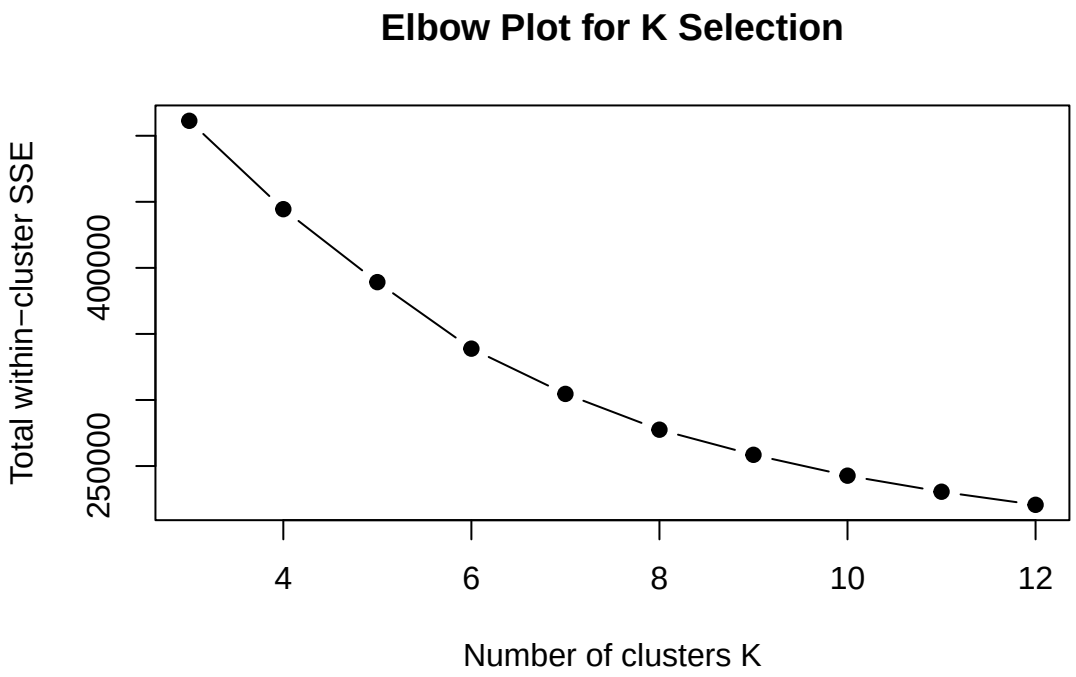
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

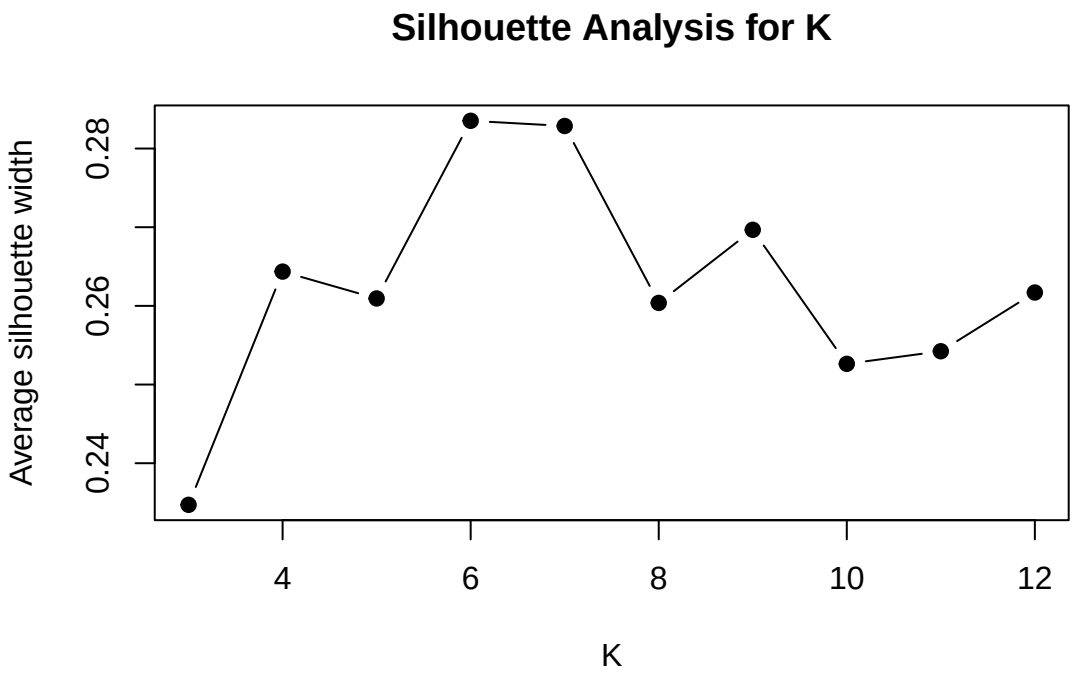
```r
plot(k_range, wcss, type="b", pch=19,
     xlab = "Number of clusters K",
     ylab = "Total within-cluster SSE",
     main = "Elbow Plot for K Selection")
```




```
idx_sub <- sample(1:nrow(scores_list), 4000)
scores_sub <- scores_list[idx_sub, ]

sil_vals <- sapply(3:12, function(k){
  km <- kmeans(scores_sub, centers = k, nstart = 50, iter.max = 1000)
  ss <- silhouette(km$cluster, dist(scores_sub))
  mean(ss[, 3])
})

plot(3:12, sil_vals, type="b", pch=19,
     xlab = "K",
     ylab = "Average silhouette width",
     main = "Silhouette Analysis for K")
```



SILHOUETTE SCORE

We choose K=6. Silhouette clearly peaks at K = 6 and Elbow stabilizes at K = 6.

```
set.seed(123)
km_main <- kmeans(scores_list, centers = 6, nstart = 50, iter.max = 1000)
```

```
## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)

## Warning: Quick-TRANSfer stage steps exceeded maximum (= 3602550)
```

```
all_listings$main_cluster <- km_main$cluster

cluster_summary_main <- all_listings %>%
  group_by(main_cluster) %>%
  summarise(
    n = n(),
    avg_price = mean(price, na.rm=TRUE),
    avg_log_price = mean(log_price, na.rm=TRUE),
    avg_occupancy = mean(occupancy_rate, na.rm=TRUE),
    avg_accommodates = mean(accommodates, na.rm=TRUE),
    avg_beds = mean(beds, na.rm=TRUE),
    avg_bedrooms = mean(bedrooms, na.rm=TRUE),
    avg_bathrooms = mean(bathrooms, na.rm=TRUE),
    avg_amenity_count = mean(amenity_count, na.rm=TRUE),
    avg_reviews = mean(number_of_reviews, na.rm=TRUE),
```

```
    avg_review_score = mean(review_scores_rating, na.rm=TRUE),
    prop_superhost = mean(host_is_superhost, na.rm=TRUE),
    avg_host_listings = mean(host_listings_count, na.rm=TRUE)
  )
cluster_summary_main
```

INTERPRETATION

```
## # A tibble: 6 x 14
##   main_cluster     n avg_price avg_log_price avg_occupancy avg_accommodates
##          <int> <int>     <dbl>         <dbl>         <dbl>            <dbl>
## 1            1 25950      154.          4.74         0.106             2.80
## 2            2 10499      679.          6.03         0.288             8.88
## 3            3  2874      391.          5.54         0.228             2.64
## 4            4 18652      163.          4.81         0.691             2.89
## 5            5  8430      166.          4.94         0.368             3.67
## 6            6  5646      174.          4.87         0.336             3.25
## # i 8 more variables: avg_beds <dbl>, avg_bedrooms <dbl>, avg_bathrooms <dbl>,
## #   avg_amenity_count <dbl>, avg_reviews <dbl>, avg_review_score <dbl>,
## #   prop_superhost <dbl>, avg_host_listings <dbl>
```

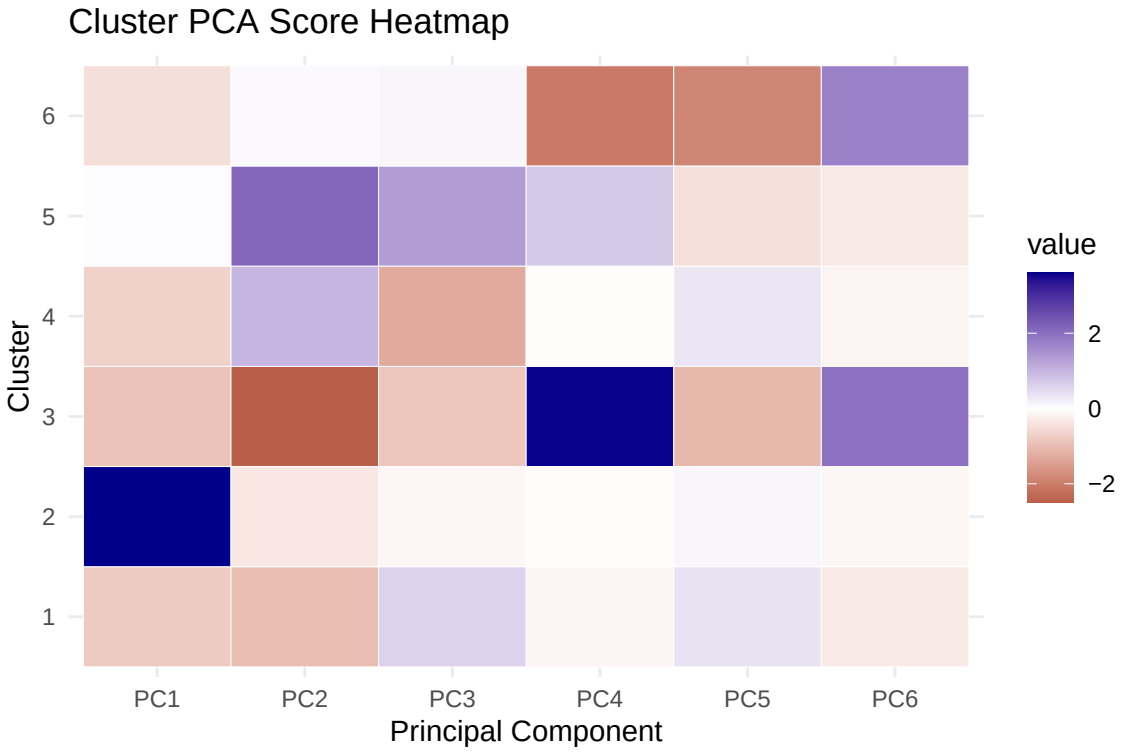
```
cluster_pca_main <- aggregate(scores_list,
                              by=list(cluster = km_main$cluster),
                              FUN=mean)

cluster_pca_main
```

```
##   cluster         PC1        PC2        PC3         PC4        PC5        PC6
## 1       1 -0.78395102 -1.0036758  0.6058232 -0.15243110  0.3807202 -0.3123460
## 2       2  3.61096467 -0.3606545 -0.1282551 -0.04503274  0.1159930 -0.1389753
## 3       3 -0.90557886 -2.4898091 -0.8532400  3.57799808 -1.0644221  1.9597504
## 4       4 -0.68093177  0.9986814 -1.2905556 -0.01962257  0.3442197 -0.1638670
## 5       5  0.05413871  2.1267728  1.3543745  0.72830430 -0.4612253 -0.3301166
## 6       6 -0.48193507  0.0764274  0.1295987 -2.05957950 -1.8722295  1.7306924
```

```
df_long <- melt(cluster_pca_main, id.vars = "cluster")
```

```
ggplot(df_long, aes(x = variable, y = factor(cluster), fill = value)) +
  geom_tile(color="white") +
  scale_fill_gradient2(low="darkred", mid="white", high="darkblue",
                       midpoint=0) +
  theme_minimal() +
  labs(title="Cluster PCA Score Heatmap",
       x="Principal Component",
       y="Cluster")
```



VISUALIZATION